

need_to_resolve.md

Contract diff between `updated_collection.json` (OpenAPI 3.0.1 — *Skygate API Documentation*, 49 paths / 57 operations, 24 `components.schemas` resources) and the Flutter client under `lib/features/` (plus `lib/core/models/` and `lib/core/constants/api_endpoints.dart`, which the features consume).

Generated: 2026-09-21 · branch `feature/checking_location_chat_issues`

Legend — **[BUG]** breaks at runtime today · **[GAP]** feature has no contract to build on · **[DOC]** collection is behind the implementation · **[DEAD]** documented but unused.

1. Request payload mismatches — these fail or silently drop data

1.1 **[BUG]** Activity feedback sends the wrong field name

POST `/api/v1/app/activities/{activityId}/feedback`

Collection	App sends
<code>rating_stars</code> (integer)	<code>rating</code>
<code>comment</code> (string)	<code>comment</code> ✓

`lib/features/journey_details/controller/cubit/activities_cubit.dart:124`

```
data: {'rating': rating, 'comment': ?comment},
```

The star rating never reaches the server (Laravel validation will 422 on the missing `rating_stars`). Rename to `rating_stars`, or confirm the backend accepts `rating`.

1.2 [DOC] Booking creation sends `hotels` and `room_type`, neither documented

POST `/api/v1/app/bookings` — documented `rooms[]` is `{package_id, locked_beds_count, pilgrims[{pilgrim_id, guardian_pilgrim_id}]}`.

- `lib/features/booking/controller/cubit/booking_cubit.dart:314-330` adds `rooms[].hotels` — a map of `{city_slug: hotel_id}`.
- `lib/features/group_booking/controller/cubit/group_booking_cubit.dart:538-558` adds `rooms[].hotels` **and** `rooms[].room_type` (slug).
- Neither cubit ever sends the documented `pilgrims[].guardian_pilgrim_id`; the guardian link is instead sent earlier on POST `app/pilgrims` (`lib/core/models/passport_data_model.dart:70`).

Decide which side is authoritative: if the server reads per-room hotel selection, the collection must document it; if not, the user's hotel choice per city is being dropped for both individual and group bookings.

1.3 [DOC] VIP / private trip request sends 3 undocumented fields and omits 1 documented one

POST `/api/v1/app/private-trip-requests` — `lib/features/vip_trip/controller/cubit/vip_trip_cubit.dart:138-158`

Field	Collection	App
<code>people_count</code> , <code>adult_count</code> , <code>child_count</code> , <code>infant_count</code>	✓	✓

preferred_start_date, preferred_end_date	✓	✓
hotel_ids	✓	✓ (omitted when empty)
requirements	✓ string	✓ (flattened multi-line string)
room_type_ids	✓ array	✗ never sent
makkah_nights	✗	sent
madinah_nights	✗	sent
rooms[] ({room_type, count})	✗	sent

The app encodes room types as slugs inside `rooms[]` + a free-text `requirements` blob instead of the documented `room_type_ids`. `PrivateTripRequestResource` also has no `rooms/makkah_nights/madinah_nights`, so a saved request cannot be read back the way it was written (see §3.4).

1.4 [DOC] Lost item report: wrong content type, doubled field names, one documented field never sent

POST `/api/v1/app/lost-items` — collection says `application/json` with `item_name`, `description`, `location`, `contact_number`.

`lib/features/sos/models/lost_item_model.dart:67-89`
sends **multipart/form-data** with: `item_name`,
`item_description`, `description`, `location`, `location_hint`,
`lost_at`, `notes`, `photo`.

- `contact_number` — **documented, never sent** (the report sheet does not collect a phone number).
- `item_description`, `location_hint`, `lost_at`, `notes`, `photo` — sent, undocumented.
- The app deliberately double-sends both spellings because the request example (`item_name/description/location`) and

LostItemResource (item_description/photo_url/location_hint) disagree **inside the collection itself**. Pin one naming down.

- photo is only possible as multipart; the collection must be updated to multipart/form-data for this operation.

1.5 [DOC] Chat message sends a duplicate message key

POST /api/v1/app/trip-chat/messages — documented body + attachment_url. lib/features/sos/models/chat_message_model.dart:97-102 sends body, message, attachment_url. message is an undocumented alias; drop it or document it.

1.6 [DOC] Register sends 3 fields absent from the documented example

POST /api/v1/auth/register — lib/features/auth/controller/cubit/register_cubit.dart:166-186

Undocumented: **full_name_ar**, **issue_place**, **issue_date**. All documented fields (full_name, mobile, password, email, passport_number, passport_expiry_date, dob, national_number, nationality, lang, gender) are sent correctly.

1.7 [DOC] Pilgrim create/update sends 4 undocumented fields

POST /api/v1/app/pilgrims and PUT /api/v1/app/pilgrims/{id} — lib/core/models/passport_data_model.dart:51-72

Undocumented: **passport_issue_date**, **passport_issue_place**, **national_number**, **guardian_pilgrim_id**. Note the collection documents the update path as the literal PUT /api/v1/app/pilgrims/1 — it should be {id}.

1.8 [GAP] Document upload has no documented body at all

POST /api/v1/app/pilgrim-documents — the collection documents **no request body**. The app posts multipart with pilgrim_id, document_type_id, document_type, file from three places:

- lib/features/profile/controller/cubit/profile_cubit.dart:314-323 (sends document_type_id and document_type)
- lib/features/auth/controller/cubit/register_cubit.dart:203-208 (sends document_type only — **no document_type_id**)
- lib/features/booking/controller/cubit/booking_cubit.dart:337-342 and lib/features/group_booking/controller/cubit/group_booking_cubit.dart:564-569 (same, document_type only)

The registration and booking flows may be uploading documents the server cannot type. Document this body and align all four call sites.

2. Endpoints the app calls that the collection does not document

Endpoint	Used by	Note
----------	---------	------

GET app/ trip- safe- area	lib/features/map/ controller/cubit/ map_cubit.dart:285	[GAP] The entire pilgrim-side geofence read. The code comments that app/trip-geofences answers 403 for pilgrims, so this route exists on the server but is missing from the collection — including its response shape, which TripGeofenceModel parses (id, name, latitude, longitude, radius_meters/radius, is_active).
POST auth/ logout	auth_cubit.dart:119	[DOC]
POST auth/ forgot- password	auth_cubit.dart:83	[DOC] body {mobile email}
POST auth/ change- password	profile_cubit.dart:149	[DOC] body {current_password, password, password_confirmation}
POST auth/ refresh	lib/core/ interceptors/	[DOC] body {refresh_token}

3. Response fields the app reads that no documented resource returns

The collection defines response *schemas* for only a handful of operations; most 200 bodies are just {data, message,

`status_code`} with data untyped. These are the concrete reads with no backing contract.

3.1 [GAP] GET `app/home` — the app's main screen has no documented response

The only example on this operation is an error payload ("`Call to protected method`

`App\Http\Repositories\BaseRepository...`", `status_code: 0`). `lib/features/home/models/home_model.dart` expects: `user, cities[] {id, country_id, country, city}, notifications[], trips {items[], meta}, vip_trips[]`. None of it is documented, and the recorded example is a 500.

3.2 [BUG-risk] Notifications: app reads `created_at`, resource returns `sent_at`

`AppNotificationResource = id, title, body, type, related_entity_type, related_entity_id, read_at, sent_at`. `lib/features/home/models/home_model.dart:78-88` reads `id, title, body ?? message, type, created_at, read_at` — **`created_at` is not in the resource and `sent_at` is never read**, so every notification timestamp is null. Also unread: `related_entity_type / related_entity_id`, which are the only way to deep-link a notification.

3.3 [GAP] GET `app/bookings/{id}` — `BookingResource` is 4 fields, the app needs a full booking

Documented: `id, status, total_amount, draft_expires_at`.

Three cubits call this endpoint and read far more:

- `lib/features/payments/controller/cubit/payments_cubit.dart:25` → `BookingPaymentModel` needs

paid_amount, currency, installments[] (amount, currency, due_at/due_date, number/sequence_order, paid_at, percentage, status) — none documented.

- lib/features/cards/controller/cubit/trip_cards_cubit.dart:60 and lib/features/journey_details/controller/cubit/segment_docs_cubit.dart:60 → TripPilgrimModel.rosterOf **needs rooms[] .pilgrims[] / booking_pilgrims[]** with pilgrim{full_name, full_name_en, passport_number, photo_url, date_of_birth}, audience, traveler_type. The pilgrim-cards and segment-documents screens cannot work off BookingResource as written.

3.4 Per-resource field diffs

PrivateTripRequestResource — read by lib/features/vip_trip/models/private_trip_request_model.dart
Undocumented reads: rooms[], room_type, count, quantity, type, makkah_hotel, madinah_hotel, makkah_nights, madinah_nights.

BookingChangeRequestResource — read by lib/features/booking_changes/models/booking_change_request_model.dart
Undocumented reads: booking, booking_number, campaign_name, trip, trip_name, trip_number. The change-request list shows the trip/booking label, which the documented resource cannot supply.

LuggageTagResource — read by lib/features/cards/models/luggage_tag_model.dart
Undocumented reads: **file_url, pdf_url**. Documented: qr_url. TripCardsCubit.downloadAll() builds its URL list from tag.fileUrl, so with only the documented fields the "download all tags" action produces an empty list and reports failure.

PilgrimTicketResource — read by PilgrimTicketModel in lib/features/journey_details/models/travel_document_model.dart
Undocumented reads: pdf_url,

number, route, segment_id, file_name.

Visas — GET app/visas has no documented response schema at all, yet VisaModel reads id, pilgrim_id, booking_pilgrim_id, visa_number/number, visa_type/type, status, file_url/pdf_url, issued_at, expires_at/expiry_date, applied_at, submitted_at. [GAP]

FinancialTransactionResource — read by lib/features/payments/models/financial_transaction_model.dart
Undocumented reads: payer, payer_name, rejection_note, rejection_reason.

PilgrimResource (id, full_name, passport_number, photo_url, linked_to_account) — lib/features/cards/models/trip_pilgrim_model.dart additionally reads full_name_en, date_of_birth/dob, audience, traveler_type. linked_to_account is documented but never read anywhere.

GET app/pilgrims/{id}/id-card — no documented response. lib/features/cards/models/pilgrim_card_model.dart reads name, name_en, passport_number, trip_number, campaign_number, departure_date/start_date_g, return_date/end_date_g, emergency_number/emergency_phone, hotels[]/stays[] (hotel_name, hotel_name_en, city, check_in(_date), check_out(_date), contact_phone, phone), card_url/file_url/pdf_url, qr_url/qr_code_url. [GAP]

GET app/document-types — no documented response. lib/features/profile/models/document_type_model.dart reads id, code, key, slug, name, label, title, type. [GAP]

Auth — POST auth/login and POST auth/register document no response. lib/features/auth/models/auth_user_model.dart needs access_token/token, refresh_token, and a user/data/pilgrim object with id, pilgrim_id, full_name/name, mobile/phone, email, avatar/photo_url. [GAP] — this is the contract the whole session depends on.

3.5 [GAP] TripDetailResource is missing activities and 40+ fields the trip screens read

TripDetailResource documents packages, hotels, itinerary, staff — but **not activities**, even though ActivityResource exists and ActivitiesCubit/MapCubit read TripService.trip(id).activities.

lib/core/models/trip_model.dart reads these keys that appear in **no** trip-related schema: activities, attendance, attendance_status, audience_labels, available_rooms, booking, booking_reference, check_in, check_out, cities_summary, current_leg, current_segment_index, due_date, duration_days, duration_in_hours, feedback_rating, filter_status, flight_number, image, installment_name, is_vip, itinerary_id, itinerary_name, min_amount_percent, my_rating, nights, packages_by_itinerary, paid_amount, payment_percentage, payment_schedules, pilgrims_count, price_range, remaining_amount, role, seats_taken, total_amount, train_ref, transport_modes, travelers_count, trip_image_url, type, vehicle, vehicle_ref.

ActivityResource similarly lacks attendance/attendance_status, my_rating/feedback_rating, and duration_in_hours, all of which drive the activities screen's attendance and rating UI.

4. Query parameters the app sends that are undocumented

Call	App sends	Collection
GET app/trips/search (home_cubit.dart:82-86)	start_date, city_id	no parameters documented

GET app/trip-chat/messages (support_chat_cubit.dart:175)	limit, before_id	no parameters documented
GET app/my-trips (trips_cubit.dart:62)	filter[status] ✓, page	only filter[status]
GET app/document-types (profile_cubit.dart:263)	none	the collection documents GET app/document- types/ required? pilgrim_id= as a separate, uncalled route

4.1 [BUG-risk] Pagination models disagree

Every documented list response uses `meta.pagination = {per_page, count, next_cursor, prev_cursor, has_more_pages}` — **cursor** based. But:

- `lib/features/trips/controller/cubit/trips_cubit.dart:62` **pages** with `page=N`, and `Meta.nextPage` (`lib/core/models/meta_model.dart`) is built from `current_page/total_pages/last_page/total` — **none of which the documented pagination block contains**. `Meta` falls back to `has_more_pages`, but no caller ever sends `cursor`, so "load more" can only ever re-fetch page 1 if the server is truly cursor-paged.
- Chat paging (`lib/features/sos/models/chat_message_model.dart`) reads `has_more` and `next_before_id` off `data` — a third, entirely undocumented pagination shape.

Pick one pagination contract and document it.

5. Documented endpoints with no caller in

lib/features [DEAD]

Endpoint	Note
GET app/notifications	The app only reads notifications embedded in app/home and calls POST app/notifications/{id}/read. The notifications sheet therefore has no paging/refresh source.
POST app/booking-change-requests	Read-only feature. booking_changes_cubit.dart only lists and shows requests — nothing in the app ever creates one, so the documented {booking_id, request_type, details} body is unimplemented.
GET app/room-assignments	No caller.
GET app/trip-segments/1/boarding-pass	No caller. Also documented with a hard-coded 1 instead of {id}.
GET app/pilgrim-tickets/{id}	Only the list route is used.
GET app/pilgrims/{id}/id-card-pdf	Only /id-card is used; PDF download goes through whatever URL that returns.
GET public/id-card/{pilgrim}	No caller (public QR landing).
POST app/financial-transactions/{id}/receipt	No caller — the receipt is attached inline on POST app/financial-transactions.
POST app/private-trip-requests/{id}/accept and /reject	No caller. vip_requests_cubit.dart implements cancel only , so a pilgrim cannot act on a returned quote.

GET/POST app/ trip-geofences, PUT app/trip- geofences/{id}, GET app/trip- geofences/ breaches	Leader-side routes; the pilgrim app uses the undocumented app/trip-safe-area instead (§2).
GET app/trips	No caller — app/home and app/my-trips cover it. (GET app/trips/{id} is used, via lib/core/services/trip_service.dart.)
GET app/ document-types/ required	No caller — the app fetches the unfiltered app/document-types.
(no route) GET app/ financial- transactions	payments_cubit.dart:38–41 getTransactions() is an empty stub — payment history is not fetched, and the collection documents no list route to fetch it from.

Also worth noting: GET app/sos-events/{id} is used (sos_cubit.dart:80), but there is **no list route** for SOS history on either side.

6. Features with no backend contract at all [GAP]

These ship from hard-coded Dart catalogues; the collection has no matching endpoint:

Feature	Source
---------	--------

Carriers (flights / trains / maritime / transport)	<code>lib/features/carriers/controller/cubit/ carriers_cubit.dart:19 → CarrierModel.catalogueOf(category); model reads id, name, logo, model, capacity from nothing</code>
Home offers	<code>lib/features/home/models/offer_model.dart (static)</code>
Hotel browse catalogue + filters	<code>lib/features/hotels/models/ hotel_catalogue.dart (static)</code>
Journey packages / routes	<code>lib/features/journey_details/models/ journey_package_model.dart, journey_route_model.dart (static)</code>
Trip offers	<code>lib/features/journey_details/models/ trip_offer_model.dart (static)</code>

`service_model.dart` and `travel_category_model.dart` have `fromJson(id, title, image/icon, description, overlay)` but no endpoint supplies them either.

7. Checked and consistent — no action needed

- `POST app/sos-events` and `POST app/location-pings` — `{latitude, longitude}` match exactly (`sos_event_model.dart:31`, `location_ping_model.dart:19`).
- `POST app/financial-transactions` — `booking_id`, `payment_method_id`, `amount`, `currency`, `reference_number`, `receipt` all match, multipart as documented (`pay_cubit.dart:96-104`).
- `POST app/trip-chat/read` — `{last_read_message_id}` matches (`support_chat_cubit.dart:136`).

- `POST app/passport-ocr/scan` — `passport_image` multipart matches at all three call sites.
 - `POST app/activities/{id}/attendance` — `{status}` matches.
 - `POST app/private-trip-requests/{id}/cancel` — matches.
 - `PaymentMethodResource`, `TripChatResource`, `TripChatMessageResource`, `SosEventResource`, `LocationPingResource`, `PilgrimDocumentResource` — every documented field is read, with only tolerant aliases added.
 - **Array-typed scalars are safe.** The collection types `status`, `details`, `admin_notes`, `instructions`, `requirements`, `resolution_notes`, `quote_details`, `address_details` as `array<string>` (almost certainly a generator artifact for Laravel `enum/text` casts). Every consumer routes through `ApiParse.labelOf / linesOf / stringsOf` (`lib/core/`
`utils/api_parse.dart:52-77`), which unwrap lists, maps and scalars alike. No code change required — but the collection should still be corrected to `string` so it stops misleading readers.
-

8. Suggested order of work

1. **§1.1** `rating_stars` — one-word fix, currently broken.
2. **§3.2** `notifications sent_at` — one-line fix, timestamps are null today.
3. **§2** document `app/trip-safe-area`, or move the map to a documented route.
4. **§3.3 / §3.5** publish the real `BookingResource` (roster + installments) and `TripDetailResource` (activities + the 40 reads) — the largest source of silent nulls.
5. **§1.4 / §1.8** settle lost-item and document-upload field naming, then align the four upload call sites.
6. **§1.2 / §1.3** decide whether per-room `hotels` and `VIP rooms[]` are real; if not, user selections are being dropped.
7. **§4.1** pick one pagination contract.
8. **§5** either wire the dead routes (create change request, accept/

reject quote, notifications list, transaction history) or drop them from the collection.

Open questions for the backend team

1. Is `app/trip-safe-area` a real route, and what does it return?
2. Does `POST app/bookings` accept `rooms[].hotels` and `rooms[].room_type`?
3. For lost items — is it `item_name/description/location` (request example) or `item_description/location_hint` (`LostItemResource`)? Does it accept multipart with `photo`?
4. Cursor or page pagination — and is there a `cursor` query parameter?
5. Is there a `GET app/financial-transactions` list route for payment history?
6. What is the actual `GET app/home` response? The only recorded example is a 500.